

# 1 Lesson 02-10-26

Last lesson we looked into *memories* and *memory interfacing*.

## 1.0.1 Addenda on memory interfacing

We introduced **masking** for memories that live within a processor's addressing space, but don't occupy it fully: these just need to consider the top  $k$  bits and output an active low enable signal.

An extra case we didn't show is that of memory combinations with different word lengths. Let's say our processor takes 16 bit words, that is it has a 16 bit data bus, and we try to populate it with a single 8 bit wordlength memory.

This means we need a more complex memory controller:

- The Data Bus is not compatible;
- We need multiple transactions.

## 1.0.2 Addenda on memory parameters

Let's name a few more memory parameters.

- We talked about **size**: this is expressed in bits, bytes or multiples of these. Requirements of this are based on the application specifications:
  - Strong impact on costs;
  - We try to reduce memory requirements;
  - Small Embedded Systems use only some Kbytes of memory.
- **Speed** can be defined in several different ways. For instance:
  - **Asynchronous reads**: from when the address given is stable to when the provided value on the data bus (by the memory chip) is stable;
  - **Asynchronous writes**: from when the address given is stable to when the provided value on the data bus (by the CPU) is guaranteed to be stored.

**Throughput** is the opposite measure, defined as the maximum number of operations per second.

For **synchronous** memories we count the time measured in clock cycles given the clock frequency.

- **Area** is a good example of a physically interesting metric: the actual size of the bit cell. The total size is given by the total number of bits multiplied by the size of a single bit cell. Over that, we need to account for the overhead of decoders, multiplexers and I/O circuits.

Note that different kind of memories have different bit cells:

- Different bit cells provide different features;
- Very strong impact on the total area;
- Very strong impact on the total cost.

In fact, memories (especially DRAMs) are at the cutting edge of technology: they are the first components to adopt smaller gate lengths;

- **Cost** is related to the size of the bit cell, the size of the memory (in number of bits), and external factors like production volume.

- Another important metric is **power consumption**: this depends on the bit cell structure and technology, and is very important for portable systems;
- **Write ability**: this refers to the manner and speed a memory can be written.
  - *High end* writeability: processor writes to memory simply and quickly. In-system, unlimited cycles;
  - *Middle range* writeability: processor writes to memory, but slower than the high end. In-system, thousands or tens of thousands of cycles;
  - *Lower range* writeability: special equipment to be used to program the memory (external programmer), thousands of cycles permitted;
  - *Low end* writeability: once at fabrication.
- **Storage permanence**, also known as its opposite, *volatility*: it's the ability of the memory to hold stored bits after they are written.
  - **High End** storage permanence: bits are never lost, even without power. Lifetime is that of the device;
  - **Middle range** storage permanence: bits are held for years, even without power. Sometimes more than the lifetime of the embedded system;
  - **Lower range** storage permanence: as long as power is supplied to the memory;
  - **Low End** storage permanence: memory content is lost after a certain amount of time, even if power is supplied. Often after a few tens of millisecond.

## 1.1 Read Only Memories

We finally come around to talking about **ROMs** (*Read Only Memories*).

Let's first consider *Mask Programmable Read Only Memories*: these are written at fabrication time, so they have:

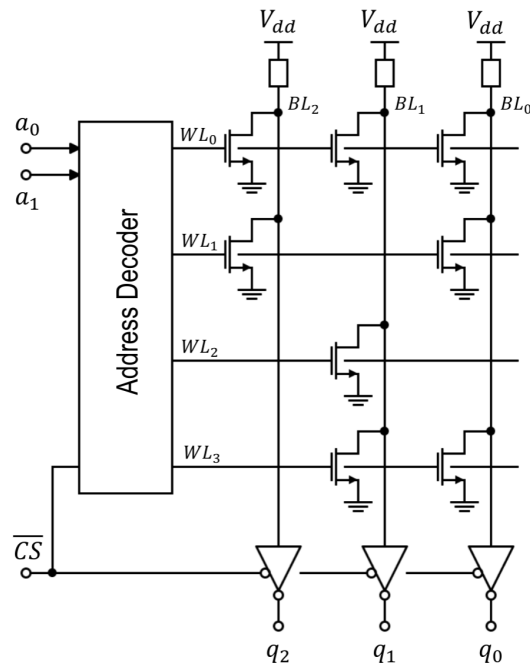
- Lowest write ability (only once)
- Highest storage permanence (for ever)

To summarize, they can only be read and not written to.

The *costs* are high NRE (we need to develop specific masks for memory contents, though the same masks may be shared for memories of the same size and organization), low per-unit, so good for very large productions.

### 1.1.1 NOR memory

A common implementation of ROM memory is the **NOR** structure.



All bit Lines are normally pulled-up to  $V_{dd}$ : the output is logic 0 because of the inverter. A single Word Line is active (high) at any given time, and transistors in the corresponding word are conducting. Transistors in all other words are in cut-off.

A conducting transistor in the active word lowers a Bit Line to 0, so the presence of a transistor at a given bit cell becomes our "one", the absence our "zero". Transistors of course are defined at the mask level: *Diffusions, Thin Oxide, Polysilicon, Metal*.